PROGRAM 3

CSC 1310 FALL 2026

FINAL PROGRAMMING PROJECT

Just like most things in life, what you get out of this class and this assignment is a direct reflection of what effort you put into it.

It is totally up to you.

This final programming project is worth 3 program grades and there are three submissions.

If you put in great effort with this project, it will be something you will be proud to add to your GitHub Repository, link to it on your resume, and talk about in future interviews!

Programming Assignment Policies

You may use Generative AI to help with

- idea generation,
- code generation, and
- code troubleshooting.

However, **all use of AI must be well documented** and links to prompts & responses **must be provided** in the required project report (described later). Failure to attribute use of AI or any other resource used is plagiarism and will result in a charge of academic integrity violation. Just to be clear, you can use generative AI to generate most or even all of the code if you understand how it works, you have tested its correctness thoroughly, it's readable and understandable, and you fully document your use (including links to your AI chat). Even if you use AI just to fix a small bug, you need to provide the link and document that.

Follow all good programming practices taught in CSC 1300 & CSC 1310.

Document your code well with comments.

Programming Project Specifications

Create a programming project of **YOUR CHOICE** that provides a useful service or implements a game with a clear objective, scoring system, and win/lose conditions. All specifications must be followed in this assignment.

Pair Programming Project

You have the choice to work on this as a pair programming project. This means you can select a partner **from your lab section**. However, no more than two students can work on the project. We can't allow you to pair up with someone who is not in your lab section, because part of your lab class time will be to work on Program 3 and because of grading logistics.

Required Data Structures

Your programming project must use at least **TWO different data structures** from the following list:

1. 2D Array
2. Doubly Linked List (can use STL list)
3. Stack (can use STL stack)
4. Queue (can use STL queue)
5. Hash Table (can use STL unordered_map)
6. Binary Search Tree (BST)
7. Graph

You may use C++ Standard Template Library (STL) containers and algorithms. You are not required to implement the data structures from scratch unless a library doesn't exist for that data structure.

If you use a binary search tree, it must be implemented using an actual (from scratch) BST (you may use example programs created by Crockett). The STL map container does not guarantee a binary search tree implementation and therefore can't be used.

Each selected data structure must:

- Have a clearly defined purpose within your application.
- Be used during normal program operation.
- Perform **at least two meaningful operations appropriate to that structure**, such as insertion, retrieval, removal, updating, or traversal.
- Support a feature that would be less natural or potentially less efficient using a different structure.

Your two required data structures do not have to interact with each other. You may use other STL containers, such as vector, to help with your program but vector doesn't count as one of your required two data structures. If you use three or more approved data structures, only two you are counting toward your minimum must meet the requirements of having two meaningful operations.

Simply declaring a data structure or filling it with data without using it meaningfully will not satisfy this requirement.

You may use additional data structures beyond the two required structures.

Examples Of Appropriate Data Structure Uses

Data Structure	Possible Applications
Stack	Undo functionality, backtracking, action history
Queue	Processing requests, managing turns, scheduling
Hash Table	Fast record lookup, inventory lookup, user accounts
Binary Search Tree	Ordered searching, sorted records, range queries
Graph	Route finding, networks, relationships, game maps
Doubly Linked List	Forward/backward navigation, playlists, browsing history, editable sequences

Important Reminder

Your selection of data structures should be based on the needs of your application rather than simply choosing the easiest structures to implement. Both required data structures must contribute meaningfully to the application's functionality. The goal is to demonstrate that you understand **when and why a particular data structure is appropriate for solving a programming problem**.

Program Requirements

Submit a fully functional C++ program that meets the requirements of this project.

Your application must:

- Provide either a useful service or a game with a clear objective, scoring system, and win/lose conditions. A "useful service" could be something like searchable movie database, recipe finder, etc. A game is expected to be entirely text based (due to complexity of creating C++ GUIs) so you don't need to worry about creating graphics or visual components outside of text. You may create a version of a game that already exists.
- Meaningfully use at least two different approved data structures.
- Load and process data from a CSV dataset (more on this in next section).
- Include at least three distinct, meaningful application features, with at least two directly using the required data structures. An example of a feature could be searching, sorting, or displaying information. Loading data from the .csv file can't count as a feature.
- Provide an interactive user interface (console-based is acceptable and expected).
- Handle invalid user input without crashing.
- Allow the user to exit the program normally.
- Use good programming practices including well documented code.

Your program can be completely menu-driven where users select numbered options if that makes sense in your application.

Project Data Set Requirements

Your project must use a data set that is saved in **.csv** format. The dataset must:

- contain at least 2,000 data records, excluding a header row labeling the columns,
- contain at least three meaningful fields per record,
- have at least three fields used within the application, and
- be loaded from the CSV file when the program runs.

Your program doesn't have to read in and process all 2000 records if it doesn't make sense for your application. Your application can read & process only the records it needs out of the data set.

In your project deliverables, you will be required to identify where you got the dataset (example: Kaggle). If you do get a public dataset (like from Kaggle), you will likely need to remove columns, clean records, or filter out some of the records.

If you created your own dataset, explain how it was generated. You may use generative AI to aid in creation of your data set so it is ok if your dataset is made of fictional data.

If you want to use more than one .csv file, that is fine, as long as they combined have at least 2000 records.

If you are creating a game then the CSV dataset can be used either for actual gameplay or used for something secondary, such as character information or scores.

Project Deliverables and Timeline

Program 3 Part 1: Due Thursday, November 12, 2026

Required Folder Organization

ZIP FOLDER

- **Project_Proposal.pdf**
- **Data/**
  - **dataset.csv**

Description of Required Files

1) **Project_Proposal.pdf**– create a document approximately 2 pages long that has the following components:

   a) Partner identification – I need to know who is working with who. You may only select one partner. You may choose to work alone. Let me know either way what you have decided to do.

   b) Goals of your project once it is completed. Provide details. What will the program do? If the program provides a service, what service will it provide? If it is a game, how will the user play/win/gain points/lose points/lose? What is the purpose? How will the data be used?

   c) List the two data structures you plan to use in your project, why those data structures will be used and what meaningful (two or more) operations will be performed with those data structures.

   d) List a description of all three planned application features.

   e) Give a chart of detailed tasks (related tasks may be grouped together) that must be accomplished (in the order they must be done) to complete the project. Include the tasks that have already been done. Include which partner has worked on each task and which partner will work on the remainder of the tasks to be done. Example:

Detailed Task Description	Person Responsible	Due Date (when this should be completed)	Current Status (not started, in-progress, done)

2) Draft data set (**.csv**) should be uploaded.

Program 3 Part 2: Due Thursday, November 19, 2026

Required Folder Organization

ZIP FOLDER

- **SourceCode/**
  - **Driver.cpp**
  - **[additional .cpp files]**
  - **[header .h files]**
- **Data/**
  - **dataset.csv**
- **README.txt**

Description of Required Files

1) Submit a functional C++ program (**.cpp and .h files**) along with the **.csv** file that contains a nearly complete programming project. Not all three features have to be working but the program should be within a few hours of being complete. Otherwise, you aren't giving yourself enough time to complete testing and documenting the project. Your README.txt (described below) should specify completely what is not done yet and what is not working. Your program can have knon bugs or incomplete features as long as they are documented in the README.

2) Submit a **README.txt** file containing the following:
   a) Project title.
   b) Names of both team members or name of single author.
   c) Description of the application's purpose. (or of the game)
   d) Instructions for compiling the program.
   e) Instructions for running and interacting with the program.
   f) Names and descriptions of the two data structures used and operations completed with those data structures.
   g) Source of the CSV dataset.
   h) Any known issues.
   i) A list of what needs to be completed before the final submission date (Part 3) and who will be completed those tasks.

Program 3 Part 3: Due Tuesday, December 1, 2026

In a **.zip** file, each person in the class must submit the files described in this section. In a pair programming submission, each partner will both submit a .zip file to canvas and their program files (**.cpp, .h, .csv**), **README.txt**, and **Final_Project_Report.pdf** will be **the exact same**.
However, each person should have their own individual/unique **Final_Project_Demonstration.mp4** and **Program 3_Report_Proof.png**.

Required Folder Organization

ZIP FOLDER

- **SourceCode/**
  - **Driver.cpp**
  - **[additional .cpp files]**
  - **[header .h files]**
- **Data/**
  - **dataset.csv**
- **Documentation/**
  - **Final_Project_Report.pdf**
  - **Final_Project_Demonstration.mp4**
- **README.txt**
- **Program 3_Report_Proof.png**

Description of Required Files

1) Final functional C++ program (**.cpp and .h files**) along with the final version of the **.csv** file.

2) Updated **README.txt**

3) **Final_Project_Demonstration.mp4** – each individual person in the class (even if they did this project as a pair programming project) must create a 2 to 5 minute **video** walkthrough demo of the project using screen casting software such as Yuja. You and your partner can not use the same recording. You each need to record yourselves independently. You do not have to show your face in the video – a screen recording with narration is sufficient. Your video must contain the following:

   a) Run your program and demonstrate its three major features.
   b) Explain how the CSV dataset is used.
   c) Identify and explain both selected data structures. Explain why the selected data structures are appropriate.
   d) Demonstrate how information is inserted, retrieved, removed, or processed.

4) **Final_Project_Report.pdf** - submit a 2 to 4-page PDF document containing the following sections. Make sure each section is formatted with a "Heading 1" style.

   a) **Project Overview** - briefly explain what your application does, who might use it, and what problem it solves or what makes the game entertaining

   b) **Dataset Description** – explain:
      i) Where the dataset came from.
      ii) How many records it contains.
      iii) What information is stored in each record.
      iv) How the application uses this information.

   c) **Data Structure Explanations** – for each of your two required data structures, explain:
      i) What data structure you selected.
      ii) What information it stores.
      iii) What functionality it supports.
      iv) Why this structure was appropriate.
      v) What other data structure you could have used and what the advantages or disadvantages would have been.
      vi) Include at least one diagram (drawing) illustrating how information is organized in your required data structures. The drawing may be a digital drawing or hand-drawn. If digital, it may be created using AI as long as AI use is documented.

   d) **Challenges & Lessons Learned** - describe at least one technical challenge you encountered and explain something you learned about choosing or using data structures.

   e) **Generative AI Usage** - identify the AI tools you used and briefly explain how you used them. Provide links to your prompts associated with this project. If you did not use generative AI at all for any reason, state that in this section.

   f) **Testing Documentation** – report at least five test cases that you tested thoroughly for your project. At least one test case must demonstrate how your program handles invalid input or an edge case. Tests should include operations involving each of the required data structures. For each test case, provide:

Test Case	Input or Action	Expected Result	Actual Result	Pass/ Fail
1	Example: Search for an existing record	Matching record displayed	Record displayed	Pass
2	Example: Search for a nonexistent record	No-match message	No-match message	Pass
3				
4				
5				

5) **Program 3_Report_Proof.png** – As with all programs, I love to try to improve this class in any way possible. I want to hear about your experience with this assignment – especially since I have never done something so open-ended before. Please fill out the report and take a screenshot to upload in your zip file.
Link: **https://tntech.co1.qualtrics.com/jfe/form/SV_3vXGh5sYqieNStM**

Partner Responsibilities & Grading

I foresee that there will be questions regarding partner responsibilities & grading so I am going to try to address what questions I think will be asked.

- Grade rubrics for all three parts are provided in a separate pdf file.
- If you work with a partner, both students will receive the same grade on the code portions of the rubric.
- You and your partner may divide the work in whatever way you both agree upon that highlights each of your strengths.
- If you fear that you will be doing all the work in the project, then don't select a partner! You will be better off taking all the credit for your work alone and it will be less hassle for you. However, if you work well with someone else in the class and you know that each of you will do your part, then it is a good idea to partner up because you could accomplish a much more significant and impressive final product and you will have more fun accomplishing it.